

Arduino und vvvv koppeln

Worum geht es in diesem Blatt?

Sie koppeln die beiden Werkzeuge der vorigen Blätter: Der Arduino liefert Messwerte, vvvv verarbeitet sie und zeigt sie an. Zuerst richten Sie die Verbindung von Hand ein, dann wechseln Sie auf das Firmata-Protokoll, das die Kopplung deutlich vereinfacht. Am Ende haben Sie eine Anwendung, die die Temperatur einer Kaffeetasse sichtbar macht.

Was erwarten wir von der Dokumentation (Protokoll)?

- Beantworten Sie die gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Machen Sie zusätzlich Screenshots Ihrer Patches/Programme und Fotos, die Ihre Lösungen in „Aktion“ zeigen (vvvv-Screenshots erstellen Sie direkt in „vvvv“ mit den Tastenkombinationen „**STRG + 1**“ bzw. „**STRG + 2**“); vergessen Sie nicht, die Patches zu speichern!
- Halten Sie Ihre **eigene Interpretation** fest: Was haben Sie im Experiment erkannt, was ist Ihnen aufgefallen?
- Welche **Probleme** sind Ihnen begegnet, und wie haben Sie sie **gelöst**?

Wie?

- **Bearbeiten Sie zur Abgabe den Test „Abgabe Praktikumsblatt 5“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie dazu Ihre Antworten, Screenshots, Fotos usw. in die jeweiligen Antwortfelder.** Das Protokoll können Sie in der Praktikumsstunde direkt im Angabe-Worddokument oder in einem separaten Textdokument schreiben.
- **Speichern Sie die vvvv-Patches und geben Sie sie als Anhänge mit ab!**
- **ACHTUNG: Chromium-basierte Browser können offenbar keine Bilder kopieren. Nutzen Sie in dem Fall einen anderen Browser, z. B. Firefox**

Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Hier kombinieren Sie die Vielseitigkeit eines Arduino-Boards (Blatt 3) mit dem Komfort und der Mächtigkeit von „vvvv“ (Blatt 2). So entwickeln Sie in kürzester Zeit Prototypen und Systeme, die mit beinahe beliebigen Ein- und Ausgabegeräten (Sensoren und Aktuatoren) funktionieren. Das eröffnet nahezu „unendliche“ Möglichkeiten für eigene kreative Projekte.

Nutzen Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Einführung in vvvv“. Sie erklären Schritt für Schritt (Tutorial-Stil) die Basisinformationen zu vvvv, Grundbegriffe, erste Programme usw. Den Umgang mit „vvvv“ erleichtern außerdem diese Ressourcen:

- Tutorial (ähnlich wie jenes der Vorlesung): `vvvvBasics - Tutorial in simple steps (English).pdf`
- Übersicht über die Tastatur- und Mausbelegung: `Leaflet dont panic - The noobs guide to vvvv.pdf`

1) Das Arduino mit „vvvv“ sprechen lassen

Wie Sie in Blatt 3 gesehen haben, lässt sich das Arduino-Board „Standalone“ verwenden: Sie entwickeln die Logik in der Arduino IDE, kompilieren sie und spielen sie auf das Board. Danach arbeitet das Board als autarkes System: Es nimmt Werte von Sensoren entgegen, verarbeitet sie und gibt Feedback/Rückmeldung über Aktoren (LEDs, Motoren, Displays usw.) an die Außenwelt.

In den folgenden Aufgaben dient das Arduino-Board als „Interface“ (Schnittstelle). Die Werte der angeschlossenen Sensoren gehen direkt an den Computer (hier: an das Programm „vvvv“) und werden dort weiterverarbeitet. Umgekehrt kann das Board Daten von einem externen Programm empfangen (hier wieder „vvvv“; natürlich gehen auch andere Programme) und an Aktoren weitergeben. Das Arduino-Board hat dabei selbst keine Logik. Das Board ist nur die Schnittstelle zwischen der physischen Welt (Sensoren, Aktoren) und der Software auf dem Computer: **Die Logik entwickeln/programmieren Sie komplett in „vvvv“.**

Für die erste Aufgabe brauchen Sie folgende Komponenten:

- PC mit installiertem „vvvv“
- Arduino UNO-Board, USB-Kabel
- Steckbrett („Breadboard“)
- Potentiometer/Drehregler, Größe 13mm, 1kOhm
- Externe LED, rot (2 Pins)

1.a) Wert eines Analogensors (Potentiometer) in „vvvv“ darstellen

Sie übermitteln den Wert eines Potentiometers/Drehreglers über das Arduino-Board an „vvvv“ und stellen ihn dort in geeigneter Form dar. Bei dieser Art des „*Interfacings*“ legen Sie auf dem Arduino-Board zuerst die Konfiguration der Ein-/Ausgänge, die Art der Datenübertragung usw. fest. Erst dann können Sie in „vvvv“ auf die Daten zugreifen.

Schritt 1: Als Erstes entwickeln Sie ein einfaches Arduino-Programm, das sich um die Kommunikation kümmert. Übertragen Sie es per „Copy & Paste“ in die Arduino IDE (oder laden Sie die Datei aus *Moodle* bzw. dem *Datenaustausch-Laufwerk „P:“*), kompilieren Sie es und laden Sie es auf das Arduino-Board. Ergänzende Erklärungen:

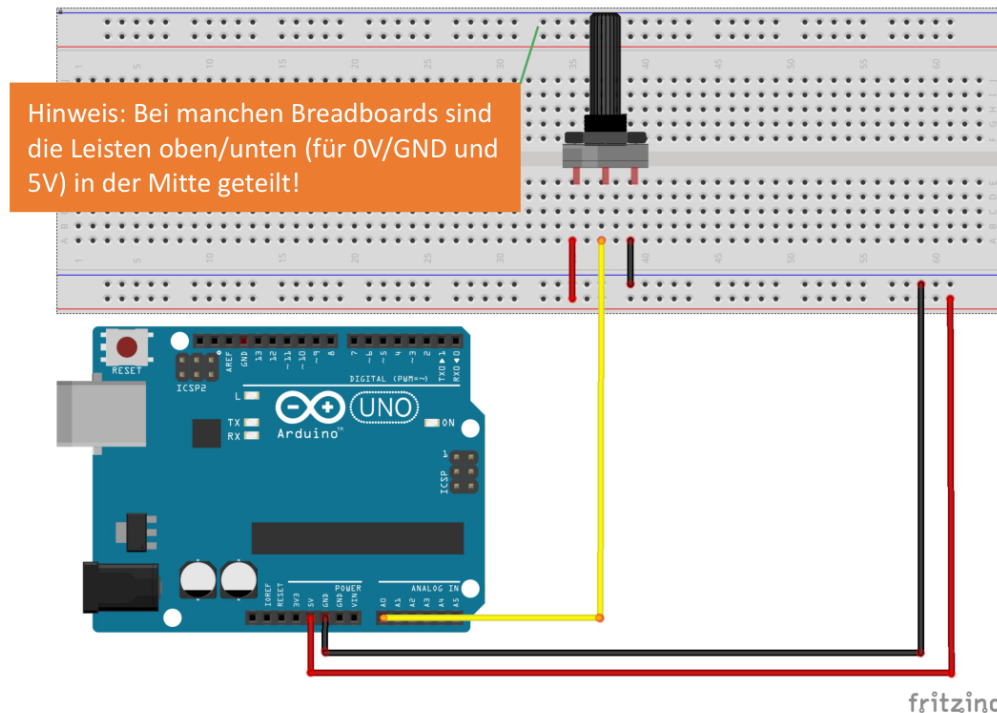
- Für die Kommunikation nutzen Sie die serielle Schnittstelle (die Sie in Blatt 3 schon zum Debugging genutzt haben): `Serial.begin(9600)`; Die Baud-Rate (9600) gibt an, wie viele Symbole pro Sekunde übertragen werden; 9600 reicht in den meisten Fällen aus
- Den Inhalt der Variablen `sensorValue` geben Sie wieder mit der Methode `Serial.println(sensorValue)`; auf die serielle Schnittstelle aus

```
/*  
Arduino als Interface für vvvv konfigurieren  
  
modified October 15, 2021  
by Andreas Riener, THI  
*/  
  
// Deklaration globaler Variablen  
int sensorPin = A0;  
int sensorValue = 0;
```

```
void setup() {  
  // Serielle Verbindung mit 9600 baud zur Kommunikation mit vvvv  
  Serial.begin(9600);  
}  
  
void loop() {  
  sensorValue = analogRead(sensorPin);  
  Serial.println(sensorValue);  
  delay(10);  
}
```



Schritt 2: Bauen Sie eine einfache Schaltung wie unten dargestellt auf. Wenn Sie den Regler drehen, ändert sich sein Widerstandswert. Der aktuelle Wert (er entspricht der Position des Drehreglers) wird am analogen Eingang „A0“ des Arduino-Boards eingelesen.



Schritt 3: Als nächstes starten wir „vvvv“ und entwickeln die Logik, um Daten vom Arduino-Board zu

Schritt 3: Als Nächstes starten Sie „vvvv“ und entwickeln die Logik, die Daten vom Arduino-Board empfängt. Zentrales Element im „vvvv“-Patch ist der „RS232 (Devices)“-Node. Er empfängt die Werte des Arduino-Boards auf der seriellen Schnittstelle. Damit die Kommunikation funktioniert, muss die sog. „Baud-Rate“, mit der das Arduino-Board sendet, mit der in „vvvv“ eingestellten übereinstimmen. Unten sehen Sie das „vvvv“-Programm, das Sie jetzt nachbauen/selbst implementieren.

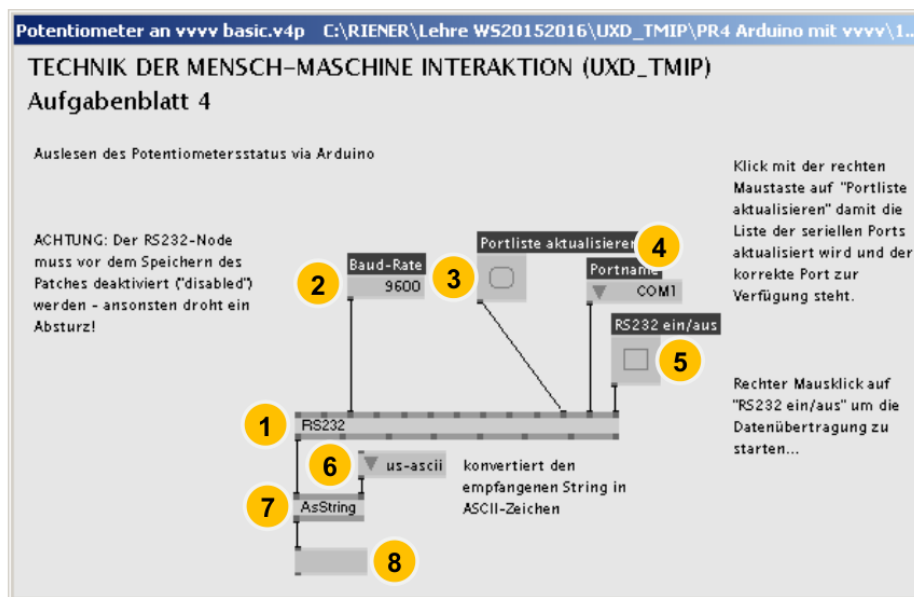
Vorgangsweise

- Starten Sie „vvvv“ und legen Sie einen neuen, leeren Patch an
- Fügen Sie nach und nach die einzelnen Komponenten („Nodes“) wie folgt ein (siehe Skizze unten!):
- **Hinweis: Die Nummern in Klammern entsprechen den Zahlen in den Kreisen der Patch-Screenshots**

(1) Ein Doppelklick (links) in das Fenster öffnet den „Node-Inspector“. Er listet alle verfügbaren „Nodes“ (Komponenten) auf; über das Eingabefeld können Sie filtern. Suchen Sie hier den „RS232 (Devices)“-Node und fügen Sie ihn in Ihren Patch ein. Vergrößern Sie den Node, damit Sie seine Optionen besser erkennen und leichter Verbindungen herstellen können

- **Hinweis (Erinnerung):** Für jeden Node gibt es im „Node-Inspector“ eine **Hilfeseite**. Ein **Klick mit der rechten Maustaste auf den Node** im „Node-Inspector“ öffnet sie, oder direkt im **Patch: Node markieren und F1-Taste**

- **Hinweis (Erinnerung) 2:** Die einzelnen Nodes konfigurieren Sie am leichtesten im „Herrn Inspektor“ („vvvv“-Hauptmenü mit der mittleren Maustaste öffnen, dort im linken, unteren Quadranten „Inspektor“ wählen)
- **Hinweis (Erinnerung) 3:** Weißer Text auf dunklem Hintergrund ist ein Beschreibungstext, den Sie zu jedem „Node“ als Kommentar angeben können (→ im „Herrn Inspektor“, Attribut „Descriptive Name“; er hat keine Funktion und dient nur zur Information)



Weitere Elemente („Nodes“) fügen Sie mit einem Doppelklick links in das „vvvv“-Programmfenster („Patch“) ein. Konfiguration inkl. Datentypen der einzelnen Elemente:

- (2) Node-Typ „IOBox (Value Advanced)“, im Inspektor als Value Type „Integer“ auswählen, als Y Input Value „9600“ eingeben
- (3) Node-Typ „IOBox (Value Advanced)“, Value Type „Boolean“, Behavior „Bang“
- (4) Node-Typ „IOBox (Enumerations)“, sobald Sie diesen Node mit dem „RS232“-Node verbinden (Parameter: Port Name), füllt er sich mit den möglichen Optionen
- (5) Node-Typ „IOBox (Value Advanced)“, Value Type „Boolean“, Behavior „Toggle“, Default „0“
- (6) Node-Typ „IOBox (Enumerations)“, sobald Sie diesen Node mit dem „As String“-Node (7) verbinden (Parameter: Encoding), füllt er sich korrekt
- (7) Node-Typ „AsString (Raw)“, Encoding „us-ascii“ (eingestellt über (6))
- (8) Node-Typ „IOBox (String)“

Hinweis: Der „RS232“-Node liefert die Werte als sogenannte „Byte-Sequenz“. Damit sie „lesbar“ werden, wandeln die Nodes 6+7 diesen Datenstrom in einen ASCII-String um, und Node (8) zeigt ihn an

- Speichern Sie den Patch als „Potentiometer an vvvv basic.v4p“ und starten Sie erst **DANACH** das Programm (den Patch) mit Rechtsklick auf (5) „RS232 ein/aus“. Im Ausgabefeld (8) sollten Sie jetzt den Wert des Potentiometers sehen

Frage 1: Fällt Ihnen an der Ausgabe etwas auf? Was? Was könnte der Grund für dieses Verhalten sein (interpretieren Sie in eigenen Worten)?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

- Um die Probleme der Basisvariante besser zu verstehen, sehen Sie sich jetzt die verbesserte Variante der Schaltung unten an. Den fertigen Patch finden Sie als „Potentiometer an vvvv complete.v4p“ im Laufwerk „P:“ (Datentausch) bzw. in *Moodle*. Experimentieren Sie mit der Schaltung und versuchen Sie zu verstehen, was die erweiterte Version macht: Sie enthält mehrere Verbesserungen...



Frage 2: Beschreiben Sie in eigenen Worten, was die erweiterte Version genau macht. An welchen Stellen wurde sie verbessert? Wie? Wie wirken sich die Verbesserungen im Programm aus? Einige Tipps dazu:

1. Entfernen Sie ruhig einzelne Teile der Schaltung, z. B. den „Tokenizer“- oder „Clean“-Node, oder ändern Sie die Konfiguration...
2. Sehen Sie sich die Hilfeseiten der einzelnen Komponenten/Nodes an!

Vergessen Sie die Dokumentation nicht: Screenshots, Fotos Ihrer Schaltungen und v4-Patches gehören ins Protokoll ;-)

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

2) Vereinfachte Interaktion mittels der „Firmata“

Wie Sie in Blatt 3 gesehen haben, lässt sich das Arduino-Board „Standalone“ verwenden: Sie entwickeln die Logik in der Arduino IDE, kompilieren sie und spielen sie auf das Board. Danach arbeitet das Board autonom, also ohne Computer. Für interaktive Anwendungen eignet sich das natürlich nicht. In Aufgabe 1 (oben) haben Sie gesehen, dass Sie Sensorwerte auch über das Arduino-Board an „vvvv“ übermitteln können. In der Standardversion (mit dem „RS232 (Devices)“-Node) ist das aber nicht besonders flexibel: Jedes neue Projekt und jede kleine Änderung erfordert eine angepasste Konfiguration des Arduino-Boards.

Mit dem Firmata-Sketch und einem speziellen „FirmataBoard“-Node betreiben Sie ein Arduino-Board (in „vvvv“) dagegen als flexibles und sehr mächtiges Interface. Wie das geht, sehen Sie in der nächsten Aufgabe.

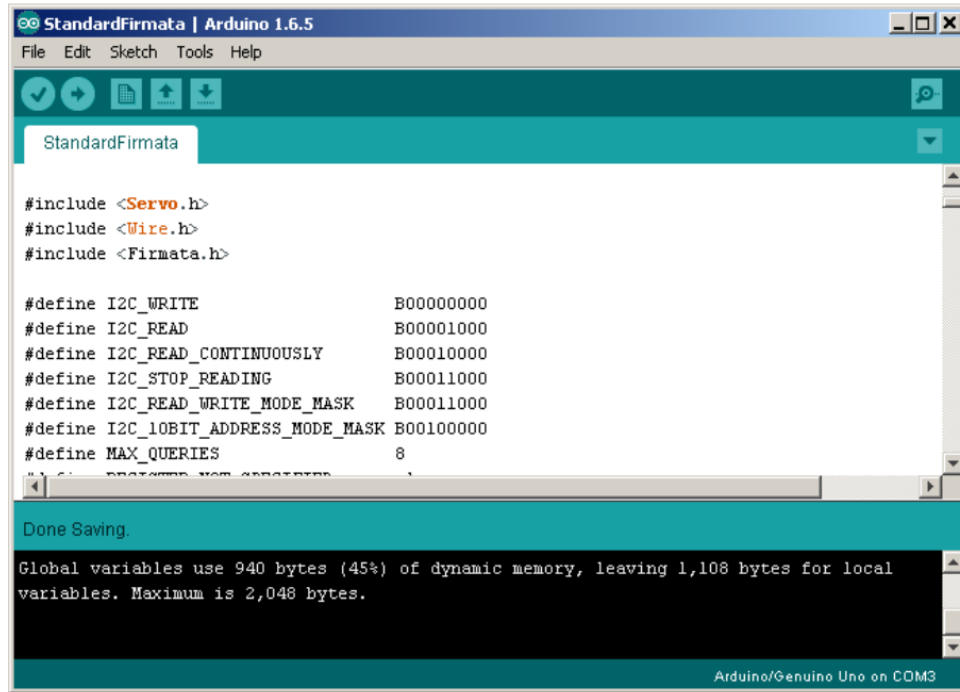
Was ist die Firmata eigentlich?

Die „Firmata“ ist ein generisches Protokoll, über das Mikrocontroller mit beliebiger Software auf dem Computer kommunizieren. Ohne sie müssten Sie dafür eigene Programme auf dem Arduino-Board schreiben, die genau angeben, welche Pins angeschlossen sind und welche Funktionen sie nutzen. Außerdem müssten Sie sicherstellen, dass die Empfängerseite das Datenpaket richtig dekodiert, damit sie mit den empfangenen Werten arbeiten kann.

Mit der „Firmata“ greifen Sie bequem auf Sensoren und Aktoren des Mikrocontrollers zu, ohne sich um Kommunikation und Codierung kümmern zu müssen.

Vorbereitung: Firmata auf das Arduino-Board laden

Der große Vorteil des „FirmataBoard“-Nodes in „vvvv“: Mit der „Firmata“ müssen Sie sich nicht mehr um die Kommunikation zwischen Arduino-Board und „vvvv“ kümmern. Laden Sie die Standard-Firmata (die Datei „StandardFirmata.ino“ liegt in Moodle bzw. im Datentausch-Laufwerk „P:“) über die Arduino IDE auf das Arduino UNO-Board: Arduino IDE starten, „Firmata“-Datei über „File“ → „Open“ öffnen und über „Sketch“ → „Upload“ auf das Arduino-Board übertragen. Ab jetzt steuern Sie das Board bequem von „vvvv“ aus an, Sie „fernsteuern“ es quasi.



```

StandardFirmata | Arduino 1.6.5
File Edit Sketch Tools Help

StandardFirmata

#include <Servo.h>
#include <Wire.h>
#include <Firmata.h>

#define I2C_WRITE          B00000000
#define I2C_READ           B00001000
#define I2C_READ_CONTINUOUSLY B00010000
#define I2C_STOP_READING  B00011000
#define I2C_READ_WRITE_MODE_MASK B00011000
#define I2C_10BIT_ADDRESS_MODE_MASK B00100000
#define MAX_QUERIES        8

Done Saving.

Global variables use 940 bytes (45%) of dynamic memory, leaving 1,108 bytes for local
variables. Maximum is 2,048 bytes.

Arduino/Genuino Uno on COM3
  
```

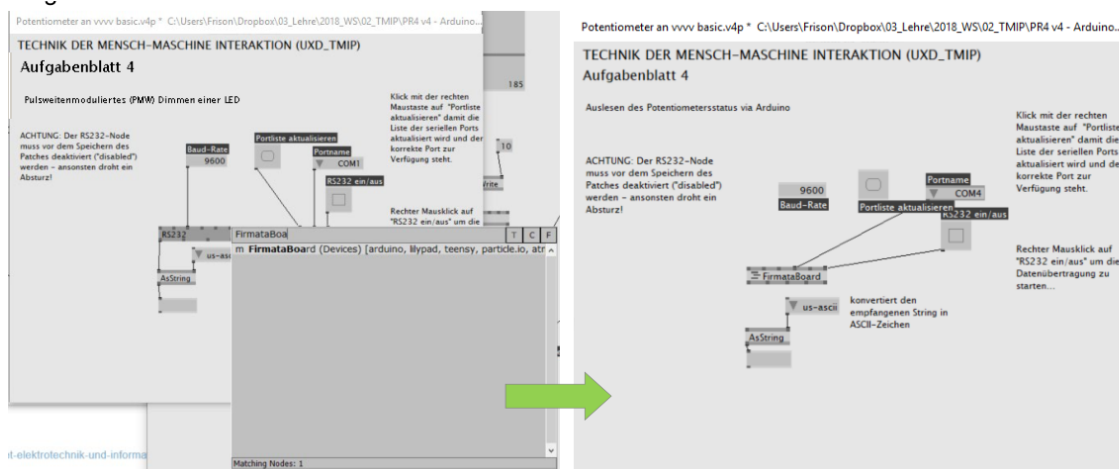
Im Folgenden geht es um einige Schaltungen, die Sie schon im Arduino-Aufgabenblatt gelöst haben, dort durch Programmieren des Arduino-Boards. **Jetzt implementieren Sie die Logik ausschließlich in „vvvv“.** Am Arduino-Board selbst (bzw. im Arduino-Code) ändern Sie nichts mehr!

2.a) Pulsweitenmoduliertes (PWM) Dimmen einer LED über „vvvv“

Schritt 1: Bauen Sie die einfache Schaltung unten (wieder) auf. Sie erinnern sich vielleicht: Die Schaltung (rote LED + Vorwiderstand 220 Ohm) ist identisch mit der aus Blatt 3.



Schritt 2: Für die Logik kopieren Sie zuerst das Programm „Potentiometer an vvvv basic.v4p“ aus Aufgabe 1, benennen es um in „FirmataBoard LED PWM.v4p“ und öffnen es in „vvvv“. Ersetzen Sie den „RS232“-Node durch einen „FirmataBoard“-Node: Doppelklick auf den „RS232“-Node, „Firmata“ ins Suchfenster eingeben, die (erste) Option „FirmataBoard (Devices)“ auswählen. Das Ergebnis sollte etwa so aussehen wie in der Abbildung unten rechts.

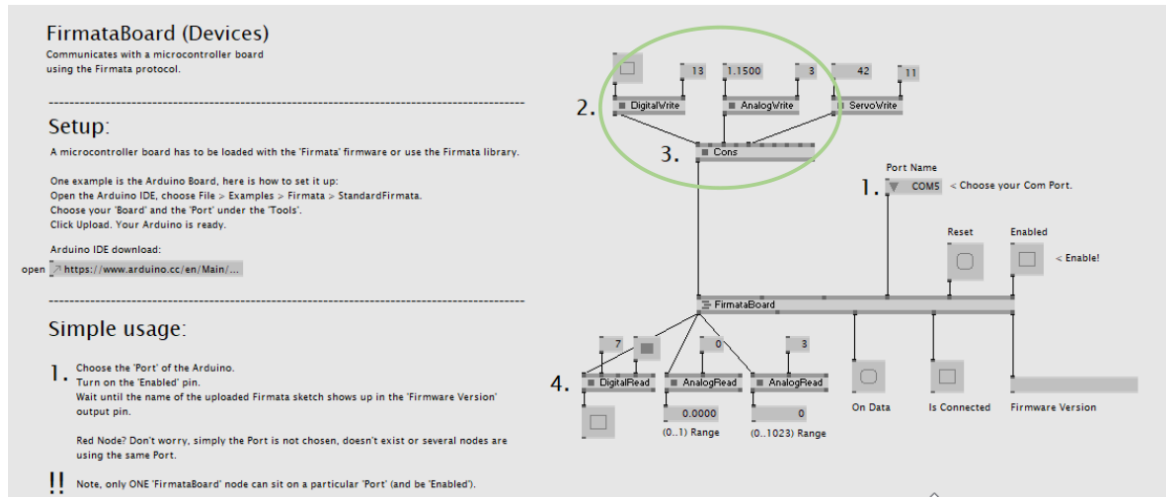


Vergrößern Sie den „FirmataBoard (Devices)“-Node, löschen Sie die Nodes „Baud-Rate“, „AsString“ und die beiden „IOBox“ („us-ascii“, leere Box ganz unten), und hängen Sie die Nodes „Portliste aktualisieren“, „Portname“ und „Enabled“ an den richtigen Stellen an den „FirmataBoard“-Node (wenn Sie die Maus über die kleinen Quadrate am „FirmataBoard“-Node bewegen, sehen Sie, welche Funktion damit verknüpft ist). Damit es übersichtlicher wird, können Sie die einzelnen Nodes noch verschieben.

Mit dem „Cons (Firmata)“-Node fassen Sie AnalogWrite und DigitalWrite des Arduino-Boards zusammen und verbinden das Ergebnis mit dem 1. Eingangspin des „FirmataBoard (Devices)“. **Das Ergebnis zeigt die Abbildung auf der nächsten Seite;** Ihr Zwischenstand sollte ungefähr gleich aussehen ;-)



Jetzt definieren Sie die Pins des Arduinos, die Sie verwenden wollen, so wie Sie es auch direkt in Arduino tun würden bzw. getan haben. Dafür gibt es „AnalogWrite“- und „DigitalWrite“-Nodes, die Sie mit dem „Cons (Firmata)“-Node konkatenieren und an das „FirmataBoard (Devices)“ anschließen. Sehen Sie sich dazu die Hilfe zum „FirmataBoard (Devices)“ bzw. „Cons (Firmata)“-Node an:



Frage 3: Wie bringen Sie jetzt eine LED an Pin 9 zum Leuchten (einfaches Ein- und Ausschalten mit einer IOBox in v4)? Bauen Sie die Schaltung auf dem Breadboard auf und erweitern Sie den „vvvv“-Patch um die notwendigen Nodes (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 4: Jetzt soll v4 eine weitere LED unabhängig von der ersten „dimmen“ (Helligkeitswert analog steuern). Wie müssen Sie Ihre Arduino-Schaltung ergänzen? Welchen Arduino-Pin verwenden Sie für die 2. LED? Wie müssen Sie den „vvvv“-Patch verändern? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

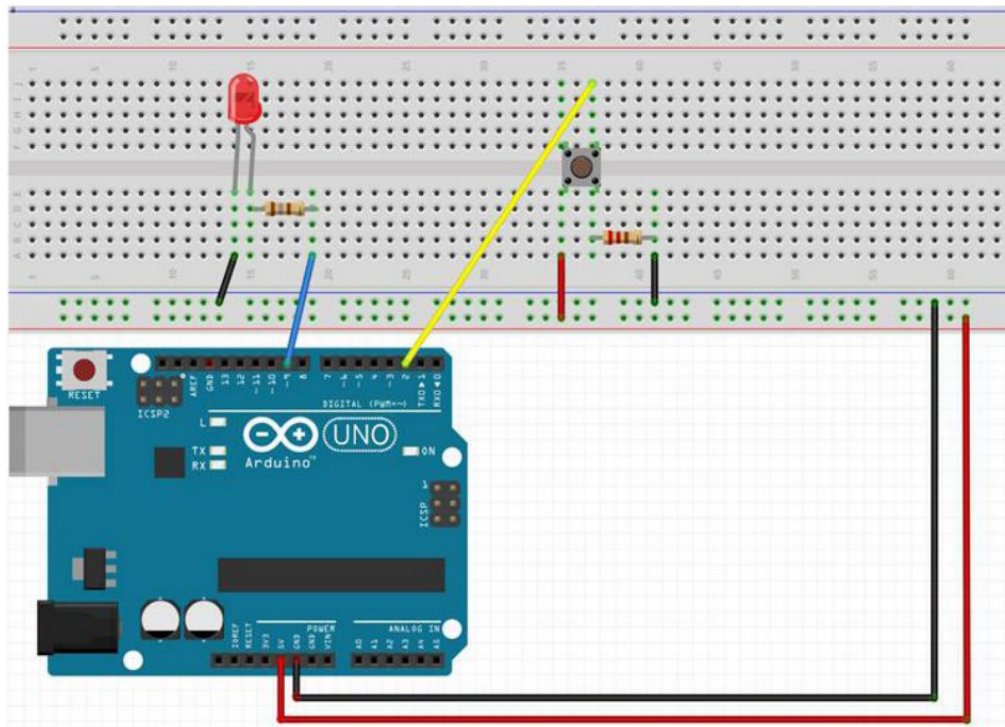
Hinweis zu „vvvv“ und Arduino über Firmata: Im Hintergrund tauschen „vvvv“ und das Arduino-Board über die serielle (USB-)Schnittstelle Konfigurationen aus. Deshalb kann es bis zu 10 Sekunden dauern, bis ein „vvvv“-Patch tatsächlich mit dem Arduino-Board kommuniziert. Planen Sie diese Wartezeit bei Ihren Tests ein (also: seien Sie nicht „zu“ ungeduldig...). Manchmal müssen Sie auch die Portliste aktualisieren, damit die Kommunikation funktioniert.

ACHTUNG: Es darf immer nur ein Patch mit aktiviertem Firmata-Node offen/aktiv sein; sonst beeinflussen sich die Patches gegenseitig, und es kann zu seltsamem Verhalten kommen...

2.b) Verwenden eines „Buttons“ als Schalter für die LED

Weiter geht's mit Ein-/Ausgabe und Feedback: Der „Push-Button“ soll jetzt die LED ein- bzw. ausschalten (die „Standalone“-Version kennen Sie schon aus dem Arduino-Blatt 3).

Schritt 1: Ergänzen Sie die Schaltung am „Breadboard“ um den „Push-Button“ samt Verkabelung wie unten dargestellt („Push-Button“ an Eingang 2, LED an Eingangs-Pin 9, Vorwiderstand 10kOhm).



Schritt 2: Ergänzen Sie jetzt selbst die Logik in „vvvv“ so, dass die LED binär ein-/ausgeschaltet wird: nicht mehr über den Slider, sondern über das Signal Ein/Aus, on/off, gedrückt/nicht gedrückt des Tasters an Pin 2 des „Arduino“-Boards. Speichern Sie den vorherigen Patch zuerst (d. h., bevor Sie etwas ändern) als „Firmata LED PushButton.v4p“ ab. Für die Änderung müssen Sie noch Folgendes wissen bzw. berücksichtigen:

- Analog zu „DigitalWrite“ gibt es einen „DigitalRead“-Node, der Ihnen den Wert des jeweiligen Pins des „Arduino“-Boards liefert. Hängen Sie ihn an den „Pin State“-Pin des FirmataBoards. Versuchen Sie zuerst, den Status des Schalters (on/off) in einer „IOBox (Value Advanced)“ anzuzeigen...
- Für Feedback (d. h., ein Eingang des Arduino-Boards steuert das Verhalten an einem Ausgang des Arduino-Boards) müssen Sie in den Feedback-Kreis einen „FrameDelay“-Node einfügen, damit zeitliche Constraints eingehalten werden (**siehe Vorlesungs-Unterlagen/Tutorial**).

Frage 5: Wie sieht Ihre Schaltung aus? Funktioniert sie wie gewünscht? Was macht der „FrameDelay“-Node genau, und was passiert, wenn Sie ihn nicht verwenden? Wo hatten Sie Probleme? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

2.c) Verwenden eines „Drehreglers“ zum Steuern der LED-Helligkeit

Im letzten Schritt regeln Sie die Helligkeit der LED mit einem Drehregler (Potentiometer). Speichern Sie den vorherigen Patch zuerst (d. h., bevor Sie etwas ändern) als „Firmata LED Potentionmeter.v4p“ ab.

Schritt 1: Ändern Sie den Aufbau am Breadboard: Der Taster samt Kabel kann weg, der Drehregler kommt dazu. Schließen Sie ihn an Port 3 bzw. A3 des Arduino-Boards an (in vvvv lesen Sie ihn über einen „AnalogRead“-Node aus). Legen Sie dazu den linken und den rechten Pin des Drehreglers auf 0V (GND) und 5V. Der mittlere Pin liefert dann je nach Stellung des Reglers einen Wert von 0...5 Volt (in vvvv: 0...1 als Realzahl).

Schritt 2: Ändern/ergänzen Sie jetzt in „vvvv“ die Programmlogik so, dass der Wert (die Stellung) des Drehreglers die Helligkeit der LED kontinuierlich regelt (ähnlich wie in Aufgabe 2.a). Verwenden Sie einen „AnalogRead“-Node (Drehregler an Eingabeindex 3) und versuchen Sie zuerst, den Wert des Drehreglers in einer „IOBox (Value Advanced)“ anzuzeigen. Wenn das funktioniert, vervollständigen Sie die Schaltung, sodass dieser Wert die LED-Helligkeit regelt. Dazu verwenden Sie einen „FrameDelay“- und dahinter einen „AnalogWrite“-Node (Ausgabe an Index/Pin 9). Hinweis: AnalogWrite erwartet als Input entweder 0...1 (Real) oder 0...1023 (Integer).

Frage 6: Wie sieht Ihre Schaltung aus? Funktioniert sie wie gewünscht? Wo hatten Sie Probleme? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

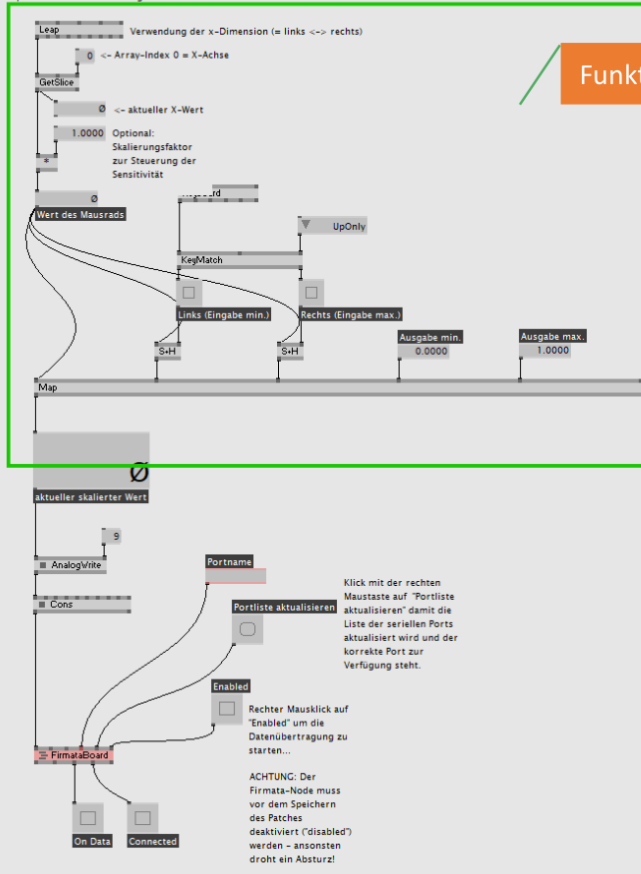
[Variante „LeapMotion“ steuert die LED-Helligkeit]

...setzen wir im Praktikum nicht um. Die Abbildung zeigt nur, wie einfach sich statt der Eingabe über Arduino/Firmata andere Eingabegeräte für die Steuerung via Arduino einsetzen lassen:

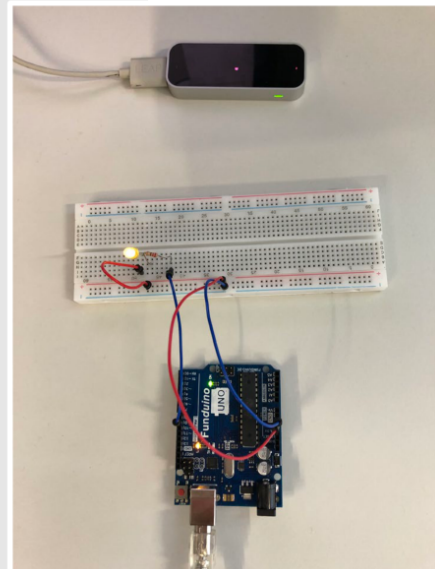
TECHNIK DER MENSCH-MASCHINE INTERAKTION (UXD_TMIP)

Aufgabenblatt 4

Leapmotion steuert LED-Helligkeit via Firmata



Funktion „LeapMotion“ aus VA03



3) Eine „sinnvolle Anwendung“: Temperaturanzeige für die Kaffeetasse

Aus der Temperatur lassen sich viele unterschiedliche Informationen ableiten. Mit einem Temperatursensor erfassen Sie diese Daten als Eingabesignal im Computer. Im letzten Beispiel dieses Aufgabenblatts verarbeitet vvvv den eingelesenen Wert eines Temperatursensors und sendet ihn an eine RGB-LED, die so die Temperatur eines Heißgetränks sichtbar macht.

3.a) Temperatursensor auslesen

Schritt 1: Ändern Sie die Schaltung auf dem Steckbrett („Breadboard“).

Zuerst muss vvvv das Signal des Temperatursensors empfangen. Dazu muss auf dem Arduino-Board wieder die „StandardFirmata“ geladen sein (falls noch nicht geschehen). In diesem Aufbau verwenden Sie einen Temperatursensor vom Typ LM35 (erinnern Sie sich an Vorführaufgabe 2: Dort hatten wir den LM35 in einer Heizungssteuerung eingesetzt. Das ist derselbe Sensor.)

Die Belegung finden Sie im Datenblatt (<http://www.ti.com/lit/ds/symlink/lm35.pdf>) oder hier:

- Temperaturbereich: -55 bis 150° Celsius (mit) bzw. **+2 bis 150° Celsius (ohne Pull-down Widerstand)**
- LM 35 liefert 0 V bei + 2°C
- Empfindlichkeit: 10 mV/° Celsius
- Anschlussschema siehe nachfolgend

Temperaturcharakteristik am Arduino-Board (mit 5V Versorgungsspannung) und in Standardbeschaltung (also +2 bis 150° Celsius); berücksichtigen Sie sie im aktuellen Beispiel:

- 10mV pro °C und 0V bei +2°C → damit bei +150°C: $0,01V \cdot (150-2) = 1,48 V / 5V = 0,296$; der max. Ausgangswert des LM35 bei Versorgungsspannung von 5V ist somit **0,296**
- **Für die Temperaturermittlung können wir somit eine Map (0.0, 0.296, 2, 150) verwenden!**

Werteabschätzung z.B. für Raumtemperatur (22° Celsius):

- 10 mV/°C und 0V bei +2°C, d.h. $10 \text{ mV} \cdot (22-2) = 0,2V / 5 = 0,040$
- Bei **Raumtemperatur (22°C)** erwarten wir also etwa einen Wert des Temperatursensors von **0,040**.

Anschlussschema:

- (1) Den rechten Pin des Sensors mit dem 5V-Anschluss des Arduino-Boards verbinden
- (2) Den mittleren Pin mit dem „Analog-In“-Port 2
- (3) Den linken Pin mit „Ground“ (0V)



Schritt 2: Ändern/erweitern Sie den Patch

Öffnen Sie den Patch „FirmataBoard TempSens.v4p“ aus dem Datenaustausch („P:“) und erweitern Sie ihn um folgende Komponenten (siehe Screenshot auf der nächsten Seite):

(1) Simulation des Drehreglers durch einen „Slider“

Grundsätzlich lesen Sie auch hier die Werte des Temperatursensors über den Arduino-Node (mit der Firmata) ein. Steht kein Arduino-Board zur Verfügung, simulieren Sie die Werte zwischen 0...1 (Real) mit einer „IOBox (Value Advanced)“, die als „Slider“ konfiguriert ist (im „Herrn Inspektor“ Slider Behavior: „Slider“, Minimum „0.0“, Maximum „1.0“, Value Type „Real“).

(2) Temperatursensor-Wert auslesen

Weil der Temperatursensor an „Pin A2“ angeschlossen ist, muss am „AnalogRead“-Node der Index 2 eingestellt sein. Ist Ihre Schaltung korrekt, kommt am Ausgangspin ein Wert zwischen 0...1 (Real) an. Achtung: Sie können **entweder** den Slider oder den Sensor verwenden!

(3) Größenkorrektur

Je nach aktueller Temperatur ist der Kreis im Renderer womöglich sehr klein. Mit einem „* (Value)“-Node und einem Multiplikationsfaktor von z. B. „5“ stellen Sie die gewünschte Größe ein.

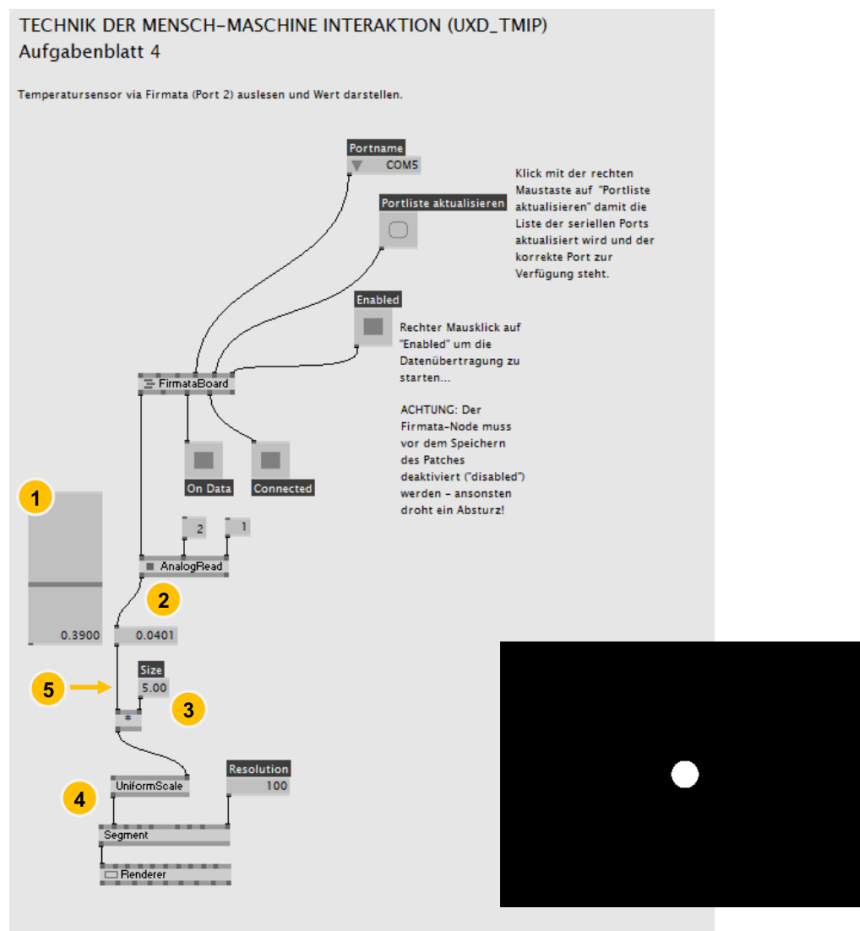
(4) Damit die Größe eines Segments den Messwert zeigt, fügen Sie einen „UniformScale (Transform)“ Node ein. Das Ergebnis zeigt anschließend ein „Renderer (EX9)“ Node an. Über den Wert von „Resolution“ stellen Sie ein, aus wie vielen Einzelsegmenten der Kreis besteht.

Speichern Sie Ihren Patch als „FirmataBoard TempSens v2.v4p“ ab.

(5) Korrekte Temperatur ausgeben

Der Ausgang des Temperatursensors zeigt bisher einen schwer interpretierbaren Wert (ca. 0,040). Rechnen Sie den Wert als Nächstes mit den Informationen zur Werteabschätzung oben in die korrekte Raumtemperatur um und steuern Sie dann mit dem Temperaturwert die Anzeige des Kreissegments an. Verwenden Sie dazu eine Map.

Speichern Sie Ihren Patch als „FirmataBoard TempSens final.v4p“ ab.



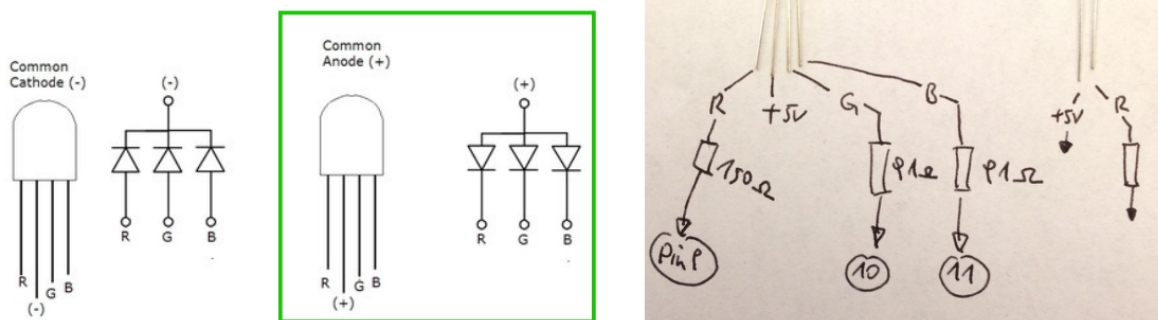
Frage 7: Wie mussten Sie den Patch erweitern, damit er die korrekte Temperatur anzeigt? Was haben Sie dabei erkannt? Sind Probleme aufgetreten? Welche?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

3.b) RGB-LED mit Temperatursensor steuern

Beachten Sie folgende Punkte:

- Die RGB-Leuchtdiode hat insgesamt 4 Pins: je einen Pin für die 3 Farbkanäle (rot, grün und blau) und einen gemeinsamen Pin für die Versorgungsspannung 5 V. Diese Bauweise heißt „*Common Anode*“ (gemeinsame Anode), im Gegensatz zu „*Common Cathode*“ (gemeinsame Kathode), siehe Skizze unten. Sie bedeutet: Die 3 kleinen Einzeleuchtdioden in der LED teilen sich den Pin für die Versorgungsspannung. Der „Plus-Pin“ (Anode) ist der längste Pin der LED, die Pins für R und G sind etwa gleich lang, und der Pin für Blau ist am kürzesten (auch das zeigt die Skizze).



Quelle: https://moderndevice.com/wp-content/uploads/2013/09/rgb_large.jpg Quelle: Prof. Dr. Andreas Riener

- Vorwiderstände ermitteln: Die Durchlass-/Sperrereigenschaften der einzelnen Leuchtdioden (R, G, B) sind unterschiedlich. Deshalb brauchen Sie auch unterschiedliche Vorwiderstände (Berechnung z. B. auf <http://www.leds.de/Widerstandsrechner>):
- **Rot:** Durchlassspannung 2.1 V, Betriebsspannung 5 V, Betriebsstrom 20 mA; $(5\text{ V} - 2.1\text{ V}) / 20\text{ mA} = 145\ \Omega \rightarrow$ nächsthöherer verfügbarer Wert = **150 Ω** (Braun, Grün, Schwarz, Schwarz)
- **Grün:** Durchlassspannung 3.7 V, ergibt 65 $\Omega \rightarrow$ nächsthöherer verfügbarer Wert = **91 Ω** (Weiß, Braun, Schwarz, Gold)
- **Blau:** Durchlassspannung 3.1 V, ergibt 95 $\Omega \rightarrow$ nächster verfügbarer Wert = **91 Ω** (Weiß, Braun, Schwarz, Gold)

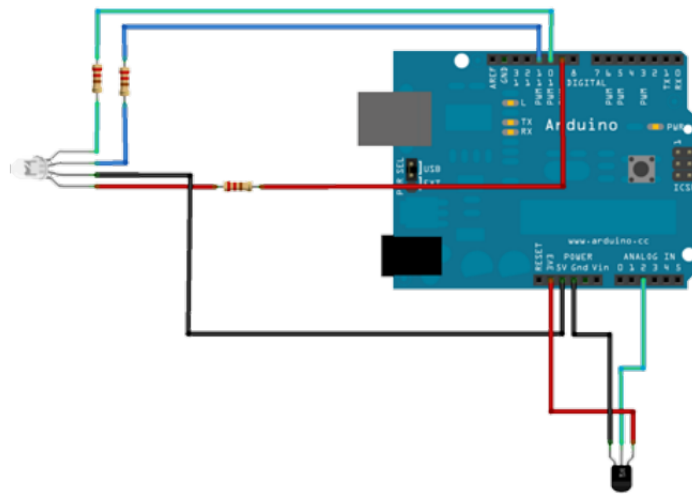
Schritt 2: Arduino-Board für die Kommunikation mit „vvvv“ vorbereiten. Die Standard-Firmata (Datei „StandardFirmata.ino“) ist schon auf dem Arduino-Board, dieser Schritt entfällt also – Juhu!

Schritt 3: Bauen Sie Ihre Schaltung nach dem Schaltplan unten (Fritzing-Skizze) auf Ihrem Breadboard nach.

Schritt 4: Steuern Sie als Nächstes die rote, grüne und blaue LED in der RGB-LED über den Wert des Temperatursensors (real bzw. simuliert) an und bringen Sie sie zum Leuchten (über Pins 9, 10, 11).

Frage 8: Wie sieht Ihre Schaltung jetzt aus? Wie verhält sie sich, wenn Sie den Wert des Temperatursensors simulieren und wenn Sie den Sensor tatsächlich verwenden? Können Sie den Wert des „realen“ Temperatursensors beeinflussen, und wie? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).



Schritt 5: Jetzt bauen Sie noch die Steuerung für die LED in das v4-Programm ein. Erweitern Sie dazu den Patch aus Aufgabe 3.a. Speichern Sie den Patch unter „FirmataBoard TempSens RGB.v4p“ ab.

- (1) Rechnen Sie den Output des Sensors auf die Hue-Werte eines gewählten Farbspektrums um. Fügen Sie dafür den Node „Map (Value)“ ein und verbinden Sie den Output des „TemperaturSensor“-Nodes mit dem Input-Pin.
- (2) Geben Sie einen minimalen und einen maximalen Wert für den Sensor-Output an und dazu einen minimalen und einen maximalen Zielwert. Dafür fügen Sie jeweils eine „IOBox (Value Advanced)“ ein.
- (3) Die Zielwerte bestimmen die Farbe der RGB-LED bei der minimalen und der maximalen gemessenen Temperatur. Mit dem Node „IOBox (Color)“ lassen Sie sich zur Orientierung einen Farbraum anzeigen. Fügen Sie außerdem eine weitere „IOBox (Value Advanced)“ ein. Im Herrn Inspektor blenden Sie mit „Show Slider“ einen Slider ein, mit dem Sie Werte festlegen.
- (4) Verbinden Sie die Ausgabe der „IOBox“ mit dem „Destination Minimum“ Pin. Fügen Sie, ebenfalls am Ausgang, einen „HSV (Color Join)“-Node ein: Damit errechnen Sie die Farbe und zeigen sie mit einer „IOBox (Color)“ an.
- (5) Wiederholen Sie das für den „Destination Maximum“-Pin. Damit sind alle nötigen Eingabewerte für den „Map (Value)“-Node definiert.
- (6) Den gemessenen Wert des Temperatursensors wandeln Sie jetzt in einen tatsächlichen Farbwert um. Dafür brauchen Sie, wie oben, einen „HSV (Color Join)“-Node. Mit einer „IOBox (Color)“ zeigen Sie ihn auch im Patch an.
- (7) Der „RGB (Color Split)“-Node ermittelt die Werte für den roten, grünen und blauen Kanal der LED.

(8) Übergeben Sie die einzelnen RGB-Werte an die Pins der RGB-LED.

(9) Verbinden Sie jetzt den Output des „FrameDelay“ Nodes mit den Input-Pins der „AnalogWrite“-Nodes.

Speichern Sie Ihren Patch als „FirmataBoard TempSens RGB.v4p“ ab.

Frage 9: Warum müssen die RGB LEDs an Pins mit PWM angeschlossen sein?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

Schritt 6: Zuletzt testen Sie, ob die Kopplung zwischen vvvv und Arduino funktioniert. Nehmen Sie eine Tasse und kochen Sie Wasser im Wasserkocher. Stellen Sie die heiße Tasse auf den Sensor und beobachten Sie, was passiert.

Frage 10: Was beobachten Sie? Welches Problem stellen Sie fest? Beschreiben Sie die Veränderung Ihrer LED.

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

Frage 11: Wie lässt sich dieses Problem beheben und Ihre Temperaturanzeige noch verbessern?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

Finale Schaltung:





BITTE BEACHTEN: Am Ende der Praktikumsstunde...

- ...bauen Sie die Schaltungen wieder ab
- ...verpacken Sie sämtliche Bauteile wieder sauber und vollständig in der Schachtel: Widerstände zurück in die richtigen Klebestreifen, Drehregler/LEDs in die Säckchen usw.
- ...verlassen Sie die Arbeitsplätze sauber und gereinigt.